An Analytical Performance Model for Streaming Tensor Programs

1 Graph Structure

A STeP program is modeled as a directed acyclic graph (DAG) G. Each STeP operator is a node; each stream connecting a producer to a consumer is an edge. We process nodes in topological order. The total execution time of the program is:

$$T_{\text{graph}} = \max_{n \in \text{leaf}(G)} \big(end(n)\big) \tag{1}$$

2 Per-Node Quantities

For each node n in the graph, we define the following quantities, derived from the operator type, its stream shapes, and the hardware parameters.

Symbol	Name	Description
$T_{\text{fire}}(n)$	Firing time	Processing time per input step (Roofline)
$N_{\text{fire}}(n)$	Firing count	Total number of firings; may be symbolic
$NIT_n^{n'}$	Input tile count	Input tiles consumed from predecessor n' per output chunk
$OTPC(n)$	Output tiles per chunk	Output tiles produced per output chunk

Table 1: Per-node quantities defined by operator type and stream shapes.

The key distinction is that T_{fire} is the cost of one *input step*, while $NIT_n^{n'}$ counts how many tiles are consumed from predecessor n' per output chunk. What constitutes a "step" depends on the operator: BinaryMap consumes one tile from each of its two inputs simultaneously per step; FlatReassemble consumes from one selected data input per step based on a control stream. For element-wise operators ($NIT = 1$), one step produces one output. For reducing operators like Accum ($NIT = K$), each output chunk requires K sequential steps, each costing T_{fire}, giving an output chunk interval of $NIT \cdot \max(T_{\text{fire}}, OTI(\text{pred}))$.

The concept of a "chunk" will come up throughout this model. An input chunk is a set of input tiles consumed to produce an output, like in the case of accum. An output chunk is a set of output tiles produced by a set of inputs, like in the case of expand. In many operations, a chunk is just one tile.

2.1 Firing Time (Roofline Model)

The firing time captures the cost of one input step. For compute operators, we use a Roofline model that accounts for compute cycles, PMU (scratchpad) store cycles, and PMU read cycles:

$$T_{\text{fire}}(n) = \max\left(T_{\text{PMU-read}}(n),\ \left\lceil \frac{FLOPs}{BW_c} \right\rceil,\ T_{\text{PMU-store}}(n)\right) \tag{2}$$

where BW_c is the allocated compute bandwidth (FLOPs/cycle).

PMU store cycles. When a compute operator writes its output tile to the on-chip scratchpad (PMU):

$$T_{\text{PMU-store}}(n) = \left\lceil \frac{S_{\text{out}}}{BW_{\text{PMU}}} \right\rceil \tag{3}$$

where S_{out} is the output tile size in bytes and $BW_{\text{PMU}} = 64$ bytes/cycle. When the output streams directly to a FIFO, $T_{\text{PMU-store}} = 0$.

PMU read cycles. When a compute operator's inputs come from PMU-resident tiles:

$$T_{\text{PMU-read}}(n) = \sum_{\substack{n' \in \text{pred}(n) \\ n' \text{ produces PMU tile}}} \left\lceil \frac{S_{n'}}{BW_{\text{PMU}}} \right\rceil \tag{4}$$

Predecessors whose output lives in streaming FIFOs contribute zero read cost.

Structural operators. For structural operators with no compute (shape operators, routing operators, Bufferize), $T_{\text{fire}}(n) = 0$.

3 Core Timing Equations

We compute the following quantities for each node n in topological order.

3.1 Start Time

A node cannot begin execution until all its input streams carry data. For source nodes (no predecessors), $st(n) = 0$.

$$st(n) = \max_{n' \in \text{pred}(n)} \big(fto(n')\big) \tag{5}$$

A consumer can start as soon as each predecessor emits its first output tile, so the start time is gated by the slowest predecessor's first-tile-out time.

3.2 First Tile Out

The absolute time at which node n produces its first output tile. Before producing output, the node must (1) wait for enough input tiles to arrive to complete its first firing, and (2) perform one firing of computation.

$$fto(n) = st(n) + ICD(n) + T_{\text{eff}}(n) \tag{6}$$

where $T_{\text{eff}}(n)$ is the effective single-firing latency:

$$T_{\text{eff}}(n) = \begin{cases} OTI_{\text{HBM}}(n) & \text{if } n \text{ is an off-chip memory op} \\ T_{\text{fire}}(n) & \text{otherwise} \end{cases} \tag{7}$$

For off-chip operators, the first-tile latency is the HBM access time rather than compute time, since these operators have no compute and the data rate is decoupled from the latency of sending data to/from HBM.

Input Chunk Delay. $ICD(n)$ is the time from when the first input tile arrives until the full input chunk for the first firing is available:

$$ICD(n) = \max_{n' \in \text{pred}(n)} \big((NIT_n^{n'} - 1) \cdot \max(T_{\text{fire}}(n),\ OTI(n'))\big) \tag{8}$$

The $\max(T_{\text{fire}}, OTI(n'))$ term reflects that each input tile consumption is gated by both the predecessor's production rate and the consumer's own processing time.

3.3 End Time

The absolute time at which node n completes all its firings.

$$end(n) = fto(n) + (N_{\text{fire}}(n) - 1) \cdot OTI(n) \tag{9}$$

After the first firing completes, the remaining $N_{\text{fire}} - 1$ output tiles arrive at steady-state intervals of OTI.

3.4 Output Chunk Interval (OCI)

The time between consecutive output chunks produced by node n.

One-shot predecessor correction. There are cases where predecessors produce only one output chunk. In such a case, propagating the OTI of the predecessor is harmful because in reality the data rates are decoupled. So when the number of firings is less than a chunk of input tiles, we discard the predecessor's OTI for purposes of calculating the current node's OCI/OTI.

Compute operators. The OCI folds the firing time into each predecessor's contribution, with the one-shot correction:

$$OCI(n) = \max_{n' \in \text{pred}(n)} \Big(NIT_n^{n'} \cdot \max\big(T_{\text{fire}}(n),\ \widetilde{OTI}(n')\big) \Big) \tag{10}$$

where the effective predecessor OTI is:

$$\widetilde{OTI}(n') = \begin{cases} T_{\text{fire}}(n) & \text{if } N_{\text{fire}}(n') \le NIT_n^{n'} \quad \text{(one-shot)} \\ OTI(n') & \text{otherwise} \quad \text{(steady-state)} \end{cases} \tag{11}$$

Off-chip memory operators. The OCI accounts for OTPC batching—a triggered load with $OTPC > 1$ issues $OTPC$ HBM requests per batch:

$$OCI(n) = \max\Big(OTPC(n) \cdot OTI_{\text{HBM}}(n),\ \max_{n' \in \text{pred}(n)} \big(NIT_n^{n'} \cdot \widetilde{OTI}(n') \big) \Big) \tag{12}$$

3.5 Output Tile Interval (OTI)

The time between consecutive output tiles produced by node n:

$$OTI(n) = \frac{OCI(n)}{OTPC(n)} \tag{13}$$

Special cases. Several operators modify the effective OTI seen by their consumers:

- **Reshape with padding**: Padding tiles are generated locally at zero cost, so the effective OTI distributes the OCI across all tiles in the chunk (both real and padding).
- **FlatPartition fan-out**: Tiles are distributed round-robin to N_c consumers, so each consumer sees tiles at $N_c\times$ the base interval.
- **FlatReassemble fan-in**: Expert outputs arrive interleaved from N_d parallel data inputs, so the effective per-predecessor OTI is divided by N_d.

4 Memory Contention Model

When multiple off-chip memory operators are active concurrently, they compete for a shared set of HBM channels. We model this with a two-pass approach and a detailed per-tile HBM access model.

4.1 Per-Tile HBM Access Model

For each off-chip operator n, the per-tile HBM access time models the pipelined dispatch and channel processing:

$$OTI_{\text{HBM}}(n) = \max(1,\ \text{bottleneck} + T_{\text{startup}} + T_{\text{sim}}) \tag{14}$$

where:

$$\text{dispatch} = \left\lceil \frac{R}{P} \right\rceil \qquad \text{(address dispatch cycles)} \tag{15}$$

$$\text{channel\_pipe} = \left(\left\lceil \frac{R_{\text{concurrent}}}{C} \right\rceil - 1 \right) \cdot I_{\text{init}} \qquad \text{(busiest channel pipeline)} \tag{16}$$

$$\text{bottleneck} = \max(\text{dispatch}, \text{channel\_pipe}) + L_{\text{ch}} \qquad \text{(pipelined overlap)} \tag{17}$$

Here R is the number of HBM requests per tile, P is the parallel dispatch width, C is the number of HBM channels, I_{init} is the per-channel initiation interval, L_{ch} is the per-channel response latency, T_{startup} is the per-tile channel startup overhead, and T_{sim} accounts for per-tile bookkeeping overhead in the simulator. $R_{\text{concurrent}}$ is the total number of concurrent HBM requests from all overlapping off-chip operators.

4.2 Two-Pass Contention Estimation

Precisely identifying which off-chip operators overlap requires knowing their time intervals, which depend on OTI_{HBM}. We resolve this circular dependency with two passes:

Pass 1 (no contention). Compute OTI_{HBM} for each off-chip operator assuming it is alone ($R_{\text{concurrent}} = R$). Run a full timing pass to estimate $[st, end]$ intervals for all nodes.

Pass 2 (with contention). For each off-chip operator n, compute the rate-weighted concurrent requests from all off-chip operators whose intervals overlap:

$$R_{\text{concurrent}}(n) = \max\left(R_n, \left\lceil \sum_{\substack{n' \in \text{offchip} \\ [st_{n'}, end_{n'}] \cap [st_n, end_n] \neq \emptyset}} R_{n'} \cdot f_{\text{overlap}}(n, n') \cdot f_{\text{rate}}(n, n') \right\rceil \right) \tag{18}$$

where f_{overlap} is the fractional temporal overlap and $f_{\text{rate}} = \min(1, OCI_n / OCI_{n'})$ scales by relative firing rate. Recompute OTI_{HBM} with the updated $R_{\text{concurrent}}$ and run a second timing pass.

5 Operator Categories

Each STeP operator defines its own values for T_{fire}, N_{fire}, NIT, and $OTPC$. The operators fall into several categories:

Shape/pass-through operators (Flatten, Promote, Reshape, Expand, Broadcast, etc.) have $T_{\text{fire}} = 0$, $NIT = 1$, $OTPC = 1$. They impose no timing cost.

Element-wise compute (UnaryMap, BinaryMap) have $NIT = 1$ per input and $OTPC = 1$. T_{fire} follows the Roofline model (Eq. 2).

Reduction operators (Accum, BinaryMapAccum) have $NIT = K$ (the reduction dimension), consuming K input tiles per output chunk. $OTPC = 1$.

Expansion operators (RepeatStatic, RetileStreamify) produce multiple output tiles per input tile. RepeatStatic has $OTPC = R$ where R is the repeat factor; RetileStreamify has $OTPC = C$ where C is the number of sub-tile chunks. Both have $T_{\text{fire}} = 0$ and $NIT = 1$.

Buffering operators (Bufferize, Streamify, DynStreamify) manage on-chip scratchpad storage. Bufferize accumulates K input tiles into a buffer ($NIT = K$, $T_{\text{fire}} = 0$). Streamify reads tiles from a buffer, producing multiple output tiles per firing ($OTPC = E$).

Off-chip memory operators (LinearOffChipLoad, OffChipStore, etc.) have $T_{\text{fire}} = 0$; their rate is determined by the HBM contention model. Source loads (no predecessor) have $OTPC = 1$. Triggered loads (LinearOffChipLoadRef) fire all tiles from one trigger at HBM speed, with $OTPC = \prod(\text{out\_shape\_tiled})$.

Routing operators (Parallelize, FlatPartition, FlatReassemble, EagerMerge) have $T_{\text{fire}} = 0$ and route tiles between producers and consumers, with fan-out/fan-in effects on the effective OTI as described in Section 4.5.

6 Handling Dynamic Dimensions

STeP streams may have dynamic-regular or ragged dimensions. These appear as symbolic variables throughout the model.

6.1 Expected-Value Substitution

For FlatPartition and FlatReassemble operators with dynamic dimensions, the model computes expected-value substitutions assuming uniform routing:

- **FlatPartition**: Expected tiles per consumer = $\lfloor N_{\text{fire}}(\text{input})/N_c \rfloor$.
- **FlatReassemble**: Expected elements per control firing = $\lfloor \sum N_{\text{fire}}(\text{data inputs})/N_{\text{fire}}(\text{control}) \rfloor$.

These substitutions are applied before the timing passes so that dynamic symbols are resolved early in rate computations.

6.2 Alternative Evaluation Strategies

The timing equations remain closed-form expressions over symbolic variables. Beyond expected-value substitution, two other strategies are available:

Concrete instantiation. Substitute symbols with values from a specific trace.

Worst/best-case bounds. Substitute extreme values to obtain performance envelopes.